Übungsblatt 2 – Lösungen

Typentheorie: Terme, Typen, Regeln

Teil A: Auf Papier

Aufgabe A1 (Universen)

Bestimmen Sie für jeden der folgenden Typen T das Universum, in dem er liegt, also das Sort w mit $\vdash T : \text{Sort } w$, und geben Sie es als Prop oder Type u an. Benutzen Sie die Regeln (Sort), (Var) und (Π-Form) aus Abschnitt 3.5 und zwei weitere Formationsregeln aus der Vorlesung: (ℕ-Form) liefert $\vdash \mathbb{N} : \text{Type}$, und nach (=-Form) ist $a = b$ für Terme a, b desselben Typs eine Aussage, also vom Typ Prop.

(a) $\mathbb{N} \to \text{Prop}$

(b) $(n : \mathbb{N}) \to n + 0 = n$

(c) $(p : \text{Prop}) \to p \to p$

(d) $(\alpha : \text{Type}) \to \alpha \to \alpha$

(e) $\text{Type} \to \text{Prop}$

(f) Nach (App) darf man in (c) für p die Aussage aus (c) selbst einsetzen, in (d) für α den Typ aus (d) dagegen nicht. Begründen Sie beides mit Ihren Ergebnissen. Welche Stelle der Regel (Π-Form) macht den Unterschied, und warum darf man bei Type nicht ebenso verfahren wie bei Prop?

Hinweis: Sort 0 = Prop und Sort $(u + 1)$ = Type u; ferner imax $u\ 0 = 0$ und sonst imax $u\ v = \max(u, v)$. Kontrollieren können Sie Ihre Ergebnisse in Lean mit `#check`.

Lösung. Nach (Sort) ist Prop = Sort 0 : Sort 1 und Type = Sort 1 : Sort 2, nach (ℕ-Form) ist $\mathbb{N} : \text{Sort } 1$. Die Regel (Π-Form) braucht die Universen Sort u der Quelle und Sort v des Ziels (im Kontext erweitert um die gebundene Variable) und liefert Sort (imax $u\ v$):

	Quelle	Ziel	Ergebnis
(a)	$\mathbb{N} : \text{Sort } 1$	$\text{Prop} : \text{Sort } 1$	Sort (imax 1 1) = Sort 1 = Type
(b)	$\mathbb{N} : \text{Sort } 1$	$n + 0 = n : \text{Sort } 0$	Sort (imax 1 0) = Sort 0 = Prop
(c)	$\text{Prop} : \text{Sort } 1$	$p \to p : \text{Sort } 0$	Sort (imax 1 0) = Prop
(d)	$\text{Type} : \text{Sort } 2$	$\alpha \to \alpha : \text{Sort } 1$	Sort (imax 2 1) = Sort 2 = Type 1
(e)	$\text{Type} : \text{Sort } 2$	$\text{Prop} : \text{Sort } 1$	Sort (imax 2 1) = Type 1

Bei (c) und (d) ist das Ziel selbst ein Pfeiltyp und wird zuerst eingeordnet. Im Kontext $p : \text{Prop}$ gilt $p : \text{Sort } 0$ nach (Var), also $p \to p : \text{Sort (imax 0 0)} = \text{Prop}$. Im Kontext $\alpha : \text{Type}$ gilt $\alpha : \text{Sort } 1$, also $\alpha \to \alpha : \text{Sort (imax 1 1)} = \text{Type}$. Bei (b) ist das Ziel nach (=-Form) eine Aussage, denn $n + 0$ und n sind Terme vom Typ ℕ.

(f) Nach (App) darf für p jeder Term vom Typ Prop stehen und für α jeder Term vom Typ Type. Die Aussage aus (c) liegt nach (c) selbst in Prop, darf also für p stehen. Der Typ aus (d) liegt dagegen in Type 1, ist also kein Term vom Typ Type und kommt für α nicht in Frage.

Den Unterschied macht imax statt max. Ist das Ziel eine Aussage ($v = 0$), so ist das Ergebnis eine Aussage, ganz gleich, wie groß die Quelle ist: In (c) läuft die Quelle über ganz Prop, und trotzdem ist das Ergebnis ein Element von Prop. Das ist die *Impredikativität* von Prop. Ist das Ziel ein Datentyp, so wird das Maximum genommen, und das Ergebnis liegt mindestens so hoch wie die Quelle: Wer über Type quantifiziert, landet in Type 1. Ginge man bei Type ebenso vor wie bei Prop, so lägen zwei impredikative Stufen übereinander, Prop und Type, und mit einer Variante von Girards Paradoxon (Abschnitt 3.1) ließe sich ein Widerspruch herleiten – die Universenhierarchie ist Notwendigkeit, nicht Vorsicht. Impredikativität allein ist nicht das Problem: Dass sie bei Prop gutgeht, ist ein Satz über den Kalkül und keine Selbstverständlichkeit.

Aufgabe A2 (Eine Typableitung)

Sei $\Gamma_0 := A : \mathrm{Prop},\ B : \mathrm{Prop},\ C : \mathrm{Prop}$.

(a) Leiten Sie mit den Regeln (Var), (λ) und (App) das Urteil

$$\Gamma_0 \vdash \lambda f.\, \lambda g.\, \lambda x.\, g\,(f\,x) : (A \to B) \to (B \to C) \to A \to C$$

her. Suchen Sie die Herleitung wie in der Vorlesung rückwärts und schreiben Sie sie als Baum auf. Abkürzungen wie $\Gamma_1 := \Gamma_0,\ f : A \to B$ sind erlaubt.

(b) Streichen Sie in Ihrem Baum die Terme. Welche Herleitung im natürlichen Schließen (Abschnitt 2.4) entsteht? Vergleichen Sie mit Ihrem Baum aus Blatt 1, Aufgabe A2 (c). Welches λ entlässt welche Annahme? Welche Einträge des Kontexts an den Blättern werden dabei zu Annahmen im natürlichen Schließen, welche nicht?

(c) Geben Sie drei verschiedene Terme vom Typ $(\alpha \to \alpha) \to \alpha \to \alpha$ an, für einen Typ α. Was liefern sie für $\alpha := \mathbb{N}$, angewandt auf die Nachfolgerfunktion und 0? Unter welchem Namen kennen Sie sie aus Abschnitt 2.4? Und was wird aus den drei Termen, wenn man für α eine Aussage $A : \mathrm{Prop}$ einsetzt?

Hinweis: Pfeile klammern nach rechts: Der Typ in (a) ist $(A \to B) \to \big((B \to C) \to (A \to C)\big)$.

Lösung. (a) Der Typ ist ein Pfeil und der Term eine Abstraktion, also war der letzte Schritt (λ). Mit den Kontexten

$$\Gamma_1 := \Gamma_0,\ f : A \to B, \qquad \Gamma_2 := \Gamma_1,\ g : B \to C, \qquad \Gamma_3 := \Gamma_2,\ x : A$$

führen drei (λ)-Schritte rückwärts auf das Ziel $\Gamma_3 \vdash g\,(f\,x) : C$. Das ist eine Anwendung, also (App) mit den Prämissen $\Gamma_3 \vdash g : B \to C$ – nach (Var), denn $g : B \to C$ steht in Γ_3 – und $\Gamma_3 \vdash f\,x : B$. Diese entsteht wieder mit (App) aus $\Gamma_3 \vdash f : A \to B$ und $\Gamma_3 \vdash x : A$, beide nach (Var). Zuerst der obere Teil, die Herleitung $\mathcal{D}$ (bei (Var) ist die Prämisse $(y : T) \in \Gamma_3$ jeweils erfüllt und nicht hingeschrieben):

$$\mathcal{D}: \quad \dfrac{\dfrac{}{\Gamma_3 \vdash g : B \to C}\,(\textsc{Var}) \qquad \dfrac{\dfrac{}{\Gamma_3 \vdash f : A \to B}\,(\textsc{Var}) \quad \dfrac{}{\Gamma_3 \vdash x : A}\,(\textsc{Var})}{\Gamma_3 \vdash f\,x : B}\,(\textsc{App})}{\Gamma_3 \vdash g\,(f\,x) : C}\,(\textsc{App})$$

Darunter die drei Abstraktionen:

$$\dfrac{\dfrac{\dfrac{\begin{array}{c}\mathcal{D}\\ \Gamma_3 \vdash g\,(f\,x) : C\end{array}}{\Gamma_2 \vdash \lambda x.\, g\,(f\,x) : A \to C}\ (\lambda)}{\Gamma_1 \vdash \lambda g.\, \lambda x.\, g\,(f\,x) : (B \to C) \to A \to C}\ (\lambda)}{\Gamma_0 \vdash \lambda f.\, \lambda g.\, \lambda x.\, g\,(f\,x) : (A \to B) \to (B \to C) \to A \to C}\ (\lambda)$$

(b) Wie auf der Folie „Terme weglassen: natürliches Schließen“ wird ohne Terme jedes (Var) zum Hinschreiben einer Annahme, jedes (App) zu →E (modus ponens) und jedes (λ) zu →I, das die Annahme der gebundenen Variablen entlässt: λf entlässt Annahme 1, λg Annahme 2, λx Annahme 3. Die Namen f, g, x spielen also die Rolle der Nummern. Es entsteht der Kettenschluss – derselbe Baum wie in Blatt 1, Aufgabe A2 (c), mit A, B, C statt φ, ψ, χ:

$$\dfrac{\dfrac{\dfrac{\dfrac{[B \to C]^2 \qquad \dfrac{[A \to B]^1 \quad [A]^3}{B}\ (\to\text{E})}{C}\ (\to\text{E})}{A \to C}\ (\to\text{I},\ 3)}{(B \to C) \to A \to C}\ (\to\text{I},\ 2)}{(A \to B) \to (B \to C) \to A \to C}\ (\to\text{I},\ 1)$$

Zu Annahmen im natürlichen Schließen werden die Einträge $f : A \to B$, $g : B \to C$ und $x : A$ von Γ_3, dem Kontext an den Blättern: Ihr Typ ist eine Aussage, im Baum stehen sie als Blätter $[A \to B]^1$, $[B \to C]^2$ und $[A]^3$, und die drei λ entlassen sie; am Ende ist keine Annahme mehr offen. Die Einträge $A, B, C : \mathrm{Prop}$ aus Γ_0 sind zwar Annahmen des Kontexts (Abschnitt 3.5), werden aber zu keinem Blatt: Ihr Typ Prop ist keine Aussage. Sie sagen nur, dass A, B und C Aussagen sind, und bleiben bis unten im Kontext. „Der Kontext sind die offenen Annahmen“ gilt also nur für die Einträge, deren Typ eine Aussage ist.

(c) Zum Beispiel $\lambda f.\, \lambda x.\, x$, $\lambda f.\, \lambda x.\, f\,x$ und $\lambda f.\, \lambda x.\, f\,(f\,x)$, allgemein $\lambda f.\, \lambda x.\, f\,(f\,(\cdots(f\,x)\cdots))$ mit n Anwendungen von f. Für $\alpha := \mathbb{N}$, angewandt auf succ und 0, liefern sie 0, 1 und 2, etwa $(\lambda f.\, \lambda x.\, f\,(f\,x))\ \mathrm{succ}\ 0 \rightsquigarrow_{\beta\beta} \mathrm{succ}\,(\mathrm{succ}\ 0) = 2$. Es sind also verschiedene Programme: die Church-Zahlen $\bar{0}$, $\bar{1}$, $\bar{2}$ aus Abschnitt 2.4, jetzt mit Typ.

Setzt man dagegen $\alpha := A : \mathrm{Prop}$, so ist $(A \to A) \to A \to A$ nach (Π-Form) eine Aussage (wie in A1 (c)), und die drei Terme sind Beweise dieser Aussage. Nach der Beweisirrelevanz (Abschnitt 3.3) sind sie definitorisch gleich. Als Programme sind es drei Geldscheine, als Beweise ist es eine Quittung.

Aufgabe A3 (Aussagen als Typen)

Seien A, B, C Aussagen. Geben Sie zu jeder der folgenden Aussagen einen Term dieses Typs an. Bausteine sind λ, Anwendung und die Terme aus den Regeln für Paare und Summen in der Vorlesung, hier für $\wedge$ und $\vee$: für $a : A$ und $b : B$ das Paar $\langle a, b\rangle : A \wedge B$, für $h : A \wedge B$ die Projektionen $h.1 : A$ und $h.2 : B$; die Injektionen $\mathrm{inl}\ a : A \vee B$ und $\mathrm{inr}\ b : A \vee B$; für $h : A \vee B$ und $u : A \to D$, $v : B \to D$ mit einer Aussage D die Fallunterscheidung $h.\mathrm{elim}\ u\ v : D$. Dabei ist $\neg A := A \to \mathrm{False}$, und ein Term für $P \leftrightarrow Q$ ist ein Paar aus einem Term für $P \to Q$ und einem für $Q \to P$.

(a) $(A \to B) \to A \wedge C \to B \wedge C$. Zeichnen Sie den Herleitungsbaum wie in der Vorlesung bei $\lambda h.\, h.2\ h.1$, an jedem Knoten sein Term.

(b) $\neg(A \vee B) \leftrightarrow \neg A \wedge \neg B$. Schreiben Sie zu jedem Baustein dazu, welcher Regel des natürlichen Schließens er entspricht.

(c) $\neg\neg\neg A \to \neg A$. Für $\neg\neg A \to A$ gibt es keinen solchen Term – hier schon.

(d) Für $\neg(A \wedge B) \to \neg A \vee \neg B$ finden Sie keinen. Erklären Sie informell, woran die Suche scheitert. Geben Sie dann einen Term an, der zusätzlich $\mathrm{em} : (P : \mathrm{Prop}) \to P \vee \neg P$ benutzen darf.

Lösung. (a) $\lambda f.\, \lambda h.\, \langle f\ h.1,\ h.2\rangle$. Der Baum, an jedem Knoten sein Term:

$$\dfrac{\dfrac{\dfrac{[f : A \to B]^1 \quad \dfrac{[h : A \wedge C]^2}{h.1 : A}\,(\wedge\mathrm{E})}{f\ h.1 : B}\,(\to\mathrm{E}) \qquad \dfrac{[h : A \wedge C]^2}{h.2 : C}\,(\wedge\mathrm{E})}{\langle f\ h.1,\ h.2\rangle : B \wedge C}\,(\wedge\mathrm{I})}{\lambda h.\, \langle f\ h.1,\ h.2\rangle : A \wedge C \to B \wedge C}\,(\to\mathrm{I},\ 2)}{\lambda f.\, \lambda h.\, \langle f\ h.1,\ h.2\rangle : (A \to B) \to A \wedge C \to B \wedge C}\,(\to\mathrm{I},\ 1)$$

Ohne die Terme ist das ein Beweis im natürlichen Schließen: Annahme 2 wird zweimal mit $\wedge$E zerlegt, mit $\to$E folgt aus $A \to B$ und A die Aussage B, $\wedge$I setzt B und C zusammen, und die beiden $\to$I entlassen die Annahmen 2 und 1. Die Namen f und h spielen die Rolle der Nummern 1 und 2; λh entlässt beide Vorkommen von h auf einmal.

(b) Hinrichtung: $\lambda h.\, \langle \lambda a.\, h\,(\mathrm{inl}\ a),\ \lambda b.\, h\,(\mathrm{inr}\ b)\rangle$. Hier sind inl und inr die Regel $\vee$I; die Anwendung von $h : \neg(A \vee B)$ ist $\to$E und liefert False; λa und λb sind $\to$I und liefern $\neg A$ und $\neg B$; das Paar ist $\wedge$I, und das äußere λh ist $\to$I. Rückrichtung: $\lambda k.\, \lambda h.\, h.\mathrm{elim}\ k.1\ k.2$. Die Projektionen $k.1 : \neg A$ und $k.2 : \neg B$ sind $\wedge$E, die Fallunterscheidung ist $\vee$E mit Ziel False, und λh, λk sind $\to$I. Der Term für die Äquivalenz ist das Paar aus beiden – $\wedge$I, wenn man $P \leftrightarrow Q$ als $(P \to Q) \wedge (Q \to P)$ liest.

(c) $\lambda h.\, \lambda a.\, h\,(\lambda n.\, n\, a)$. Aus $a : A$ entsteht $\lambda n.\, n\, a : \neg\neg A$ – das ist der Term für $A \to \neg\neg A$ –, und h macht daraus False. Hier muss am Ende nur False herauskommen, und das liefert h. Für $\neg\neg A \to A$ müsste man dagegen einen Beweis von A herstellen, und dafür hat man nichts in der Hand.

(d) Ein Term dieses Typs bekommt $h : \neg(A \wedge B)$ und muss am Ende $\mathrm{inl}\ u$ mit $u : \neg A$ oder $\mathrm{inr}\ v$ mit $v : \neg B$ liefern – er muss sich für eine Seite entscheiden. Für u müsste er aus einem $a : A$ einen Beweis von False bauen. Die einzige Quelle dafür ist h, und h verlangt einen Beweis von $A \wedge B$, also außer a auch ein $b : B$, das er nicht hat. Für v fehlt ebenso ein a. Ohne zu wissen, welche der beiden Aussagen falsch ist, kann der Term keine Seite wählen. (Dass es wirklich keinen Term gibt, zeigt man mit mehr Theorie, etwa über Normalformen; die Idee ist diese.) Mit em kann er die Entscheidung treffen:

$$\lambda h.\ (\mathrm{em}\ A).\mathrm{elim}\ (\lambda a.\, \mathrm{inr}\,(\lambda b.\, h\,\langle a, b\rangle))\ (\lambda n.\, \mathrm{inl}\ n).$$

Gilt A, so ist B falsch; gilt $\neg A$, steht die linke Seite schon da.

Aufgabe A4 (Rechnen im Kern)

Sei zweimal $:= \lambda f.\, \lambda x.\, f\,(f\,x)$ und doppelt $:= \lambda n.\, n + n$, beide für natürliche Zahlen. Schreiben Sie an jeden Reduktionsschritt die Regel (β, δ, ζ, ι). Summen und Produkte fertiger Zahlen wie $3 + 3$ dürfen Sie am Ende in einem Schritt ausrechnen.

(a) Zeigen Sie zweimal doppelt $3 \rightsquigarrow 12$ und (**let** $d :=$ doppelt; $d\,(d\,1)) \rightsquigarrow 4$.

(b) Sei $r :=$ rec $1\ s$ der Rekursor über $\mathbb{N}$ mit dem Schritt $s := \lambda _\ ih.\ 2 \cdot ih$, also $r\ 0 \rightsquigarrow 1$ und $r\,(n{+}1) \rightsquigarrow s\ n\ (r\ n)$. Zeigen Sie $r\ 3 \rightsquigarrow 8$. Welche Funktion berechnet r?

(c) In Lean sind $+$ und $\cdot$ auf $\mathbb{N}$ durch Rekursion über das *zweite* Argument definiert,

$$n + 0 \rightsquigarrow n, \qquad n + \mathrm{succ}\ m \rightsquigarrow \mathrm{succ}\,(n + m), \qquad n \cdot 0 \rightsquigarrow 0, \qquad n \cdot \mathrm{succ}\ m \rightsquigarrow n \cdot m + n,$$

und 1 ist succ 0. Welche der folgenden Gleichungen gelten für eine Variable $n : \mathbb{N}$ definitorisch, welche nur propositional? Begründen Sie durch Rechnen.

(i) $n + 1 = \mathrm{succ}\ n$ (ii) $1 + n = \mathrm{succ}\ n$ (iii) $0 \cdot n = 0$
(iv) $n \cdot 1 = n$ (v) $n \cdot 1 = 0 + n$

Hinweis: Eine durch Rekursion definierte Funktion rechnet nur weiter, wenn das Argument, über das sie rekursiert, mit einem Konstruktor beginnt. Da die Reduktion konfluent ist und stets endet, sind zwei Terme genau dann definitorisch gleich, wenn sie dieselbe Normalform haben (Abschnitt 3.5).

Lösung. (a) Erst die Definitionen auffalten, dann einsetzen:

$$\begin{aligned}\text{zweimal doppelt } 3 &\rightsquigarrow_\delta (\lambda f.\, \lambda x.\, f\,(f\,x))\ \text{doppelt}\ 3 \rightsquigarrow_\beta (\lambda x.\, \text{doppelt}\,(\text{doppelt}\ x))\ 3\\ &\rightsquigarrow_\beta \text{doppelt}\,(\text{doppelt}\ 3) \rightsquigarrow_\delta (\lambda n.\, n + n)\,(\text{doppelt}\ 3)\\ &\rightsquigarrow_\beta \text{doppelt}\ 3 + \text{doppelt}\ 3 \rightsquigarrow_{\delta\delta} (\lambda n.\, n + n)\ 3 + (\lambda n.\, n + n)\ 3\\ &\rightsquigarrow_{\beta\beta} (3 + 3) + (3 + 3) = 12.\end{aligned}$$

Hier wurde das äußere doppelt zuerst ausgewertet. Wertet man innen zuerst aus, kommt man über doppelt $6 \rightsquigarrow 6 + 6$ zum selben Ergebnis – Konfluenz. Beim **let** wird zuerst eingesetzt:

$$\begin{aligned}(\textbf{let}\ d :=\ \text{doppelt};\ d\,(d\,1)) &\rightsquigarrow_\zeta \text{doppelt}\,(\text{doppelt}\ 1) \rightsquigarrow_{\delta\beta} \text{doppelt}\ 1 + \text{doppelt}\ 1\\ &\rightsquigarrow_{\delta\delta\beta\beta} (1 + 1) + (1 + 1) = 4.\end{aligned}$$

(b) Mit $3 = \mathrm{succ}\ 2$, $2 = \mathrm{succ}\ 1$, $1 = \mathrm{succ}\ 0$; jeder Übergang $s\ n\ (r\ n) \rightsquigarrow 2 \cdot r\ n$ besteht aus zwei β-Schritten:

$$\begin{aligned}r\ 3 &\rightsquigarrow_\iota s\ 2\ (r\ 2) \rightsquigarrow_{\beta\beta} 2 \cdot r\ 2 \rightsquigarrow_{\iota\beta\beta} 2 \cdot (2 \cdot r\ 1) \rightsquigarrow_{\iota\beta\beta} 2 \cdot (2 \cdot (2 \cdot r\ 0))\\ &\rightsquigarrow_\iota 2 \cdot (2 \cdot (2 \cdot 1)) = 8.\end{aligned}$$

r berechnet $n \mapsto 2^n$: Es ist $r\ 0 = 1 = 2^0$ und $r\,(n{+}1) = 2 \cdot r\ n$, und das sind genau die Rekursionsgleichungen der Zweierpotenz.

(c) Definitorisch gelten (i) und (v), nur propositional (ii), (iii) und (iv).

(i) $n + 1 = n + \mathrm{succ}\ 0 \rightsquigarrow_\iota \mathrm{succ}\,(n + 0) \rightsquigarrow_\iota \mathrm{succ}\ n$: dieselbe Normalform wie die rechte Seite.

(ii) Die Summe $1 + n$ rekursiert über n, eine Variable; keine Regel greift. $1 + n$ und $\mathrm{succ}\ n$ sind verschiedene Normalformen. Wahr ist die Gleichung trotzdem, aber das zeigt erst eine Induktion über n.

(iii) Ebenso: $0 \cdot n$ steckt fest, die Normalform ist nicht 0. Propositional gilt die Gleichung (Induktion über n).

(iv) $n \cdot 1 = n \cdot \operatorname{succ} 0 \rightsquigarrow_\iota n \cdot 0 + n \rightsquigarrow_\iota 0 + n$, und hier steckt die Rechnung fest: Die Addition rekursiert über ihr zweites Argument, und das ist die Variable n. Die Normalform $0 + n$ ist nicht n, die Gleichung gilt also nicht definitorisch – wohl aber propositional, denn $0 + n = n$ zeigt man durch Induktion.

(v) Links steht nach (iv) die Normalform $0 + n$, rechts auch.

Bemerkenswert sind (iv) und (v) zusammen: Bis zur Gleichung $n \cdot 1 = n$ fehlt genau der propositionale Schritt $0 + n = n$. Darauf beruht B3.

Aufgabe A5 (Binärbäume)

Der induktive Typ Baum hat die Konstruktoren blatt : Baum und knoten : Baum → Baum → Baum. Durch Fallunterscheidung nach dem Konstruktor sind definiert:

$$\begin{aligned} \text{blaetter blatt} &= 1, & \text{blaetter}\,(\text{knoten } l\ r) &= \text{blaetter } l + \text{blaetter } r, \\ \text{innen blatt} &= 0, & \text{innen}\,(\text{knoten } l\ r) &= \text{innen } l + \text{innen } r + 1, \\ \text{spiegel blatt} &= \text{blatt}, & \text{spiegel}\,(\text{knoten } l\ r) &= \text{knoten}\,(\text{spiegel } r)\,(\text{spiegel } l). \end{aligned}$$

(a) Zeichnen Sie $t := \text{knoten}\,(\text{knoten blatt blatt})\,(\text{knoten blatt}\,(\text{knoten blatt blatt}))$ und berechnen Sie blaetter t, innen t und spiegel t.

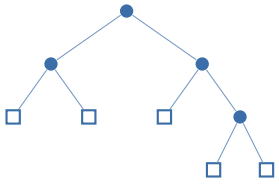
(b) Geben Sie den Typ des Rekursors Baum.rec an und seine beiden ι-Regeln. Was ergibt sich, wenn das Motiv Werte in Prop hat? Schreiben Sie blaetter als Anwendung des Rekursors.

(c) Zeigen Sie durch strukturelle Induktion: blaetter t = innen $t + 1$ für alle t : Baum.

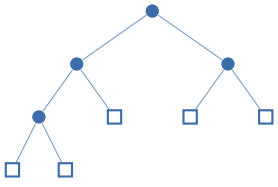
(d) Zeigen Sie ebenso spiegel (spiegel t) = t für alle t. Welche Schritte Ihres Beweises sind Rechnen, also definitorisch, und wo geht die Induktionsvoraussetzung ein?

Hinweis: Für ℕ hat der Rekursor den Typ $\text{Nat.rec} : \{\mathit{motive} : \mathbb{N} \to \text{Sort } u\} \to \mathit{motive}\ 0 \to \big((n : \mathbb{N}) \to \mathit{motive}\ n \to \mathit{motive}\,(n+1)\big) \to (t : \mathbb{N}) \to \mathit{motive}\ t$.

Lösung. (a) Innere Knoten als Punkte, Blätter als Quadrate:



t



spiegel t

Nach den Gleichungen ist blaetter $t = (1+1) + (1 + (1+1)) = 5$ und innen $t = (0+0+1) + (0 + (0+0+1) + 1) + 1 = 4$. Für spiegel t werden die beiden Teilbäume vertauscht und selbst gespiegelt; spiegel (knoten blatt blatt) = knoten blatt blatt, also

$$\text{spiegel } t = \text{knoten}\,(\text{knoten}\,(\text{knoten blatt blatt})\ \text{blatt})\,(\text{knoten blatt blatt}).$$

(b) Ein Fall pro Konstruktor; im Fall knoten gibt es zwei rekursive Argumente und deshalb zwei Werte für die Teilbäume:

$$\begin{aligned}\text{Baum.rec} : \{\mathit{motive} : \text{Baum} \to \text{Sort } u\} &\to \mathit{motive}\ \text{blatt}\\ &\to ((l\ r : \text{Baum}) \to \mathit{motive}\ l \to \mathit{motive}\ r \to \mathit{motive}\,(\text{knoten}\ l\ r))\\ &\to (t : \text{Baum}) \to \mathit{motive}\ t.\end{aligned}$$

Die ι-Regeln, mit $\mathrm{R} := \text{Baum.rec}\ b\ k$:

$$\mathrm{R}\ \text{blatt} \rightsquigarrow b, \qquad \mathrm{R}\,(\text{knoten}\ l\ r) \rightsquigarrow k\ l\ r\ (\mathrm{R}\ l)\ (\mathrm{R}\ r).$$

Ist das Motiv ein Prädikat $P : \text{Baum} \to \text{Prop}$, so wird der Rekursor zum Induktionsprinzip:

$$P\ \text{blatt} \to ((l\ r : \text{Baum}) \to P\ l \to P\ r \to P\,(\text{knoten}\ l\ r)) \to (t : \text{Baum}) \to P\ t,$$

ein Induktionsanfang für blatt und ein Induktionsschritt mit *zwei* Induktionsvoraussetzungen, eine für jeden Teilbaum.

Mit dem konstanten Motiv $\lambda_.\ \mathbb{N}$ ist blaetter = Baum.rec 1 $(\lambda\, l\ r\ x\ y.\ x + y)$; die zweite ι-Regel und vier β-Schritte, einer pro Argument, ergeben genau $\text{blaetter}\,(\text{knoten}\ l\ r) \rightsquigarrow \text{blaetter}\ l + \text{blaetter}\ r$.

(c) Induktion mit $P\ t := (\text{blaetter}\ t = \text{innen}\ t + 1)$; im Schritt gelten $P\ l$ und $P\ r$.

$$\begin{aligned}\text{Anfang:}\quad & \text{blaetter blatt} = 1 = 0 + 1 = \text{innen blatt} + 1,\\ \text{Schritt:}\quad & \text{blaetter}\,(\text{knoten}\ l\ r) = \text{blaetter}\ l + \text{blaetter}\ r = (\text{innen}\ l + 1) + (\text{innen}\ r + 1)\\ & = (\text{innen}\ l + \text{innen}\ r + 1) + 1 = \text{innen}\,(\text{knoten}\ l\ r) + 1.\end{aligned}$$

Im Schritt sind das erste und das letzte Gleichheitszeichen die Definitionen, das zweite die Induktionsvoraussetzungen, das dritte Rechnen mit Assoziativ- und Kommutativgesetz in $\mathbb{N}$ – und das ist, wie in A4 (c), *nicht* definitorisch, sondern braucht selbst Induktion.

(d) Induktion mit $P\ t := (\text{spiegel}\,(\text{spiegel}\ t) = t)$:

$$\begin{aligned}\text{Anfang:}\quad & \text{spiegel}\,(\text{spiegel blatt}) = \text{spiegel blatt} = \text{blatt},\\ \text{Schritt:}\quad & \text{spiegel}\,(\text{spiegel}\,(\text{knoten}\ l\ r)) = \text{spiegel}\,(\text{knoten}\,(\text{spiegel}\ r)\,(\text{spiegel}\ l))\\ & = \text{knoten}\,(\text{spiegel}\,(\text{spiegel}\ l))\,(\text{spiegel}\,(\text{spiegel}\ r))\\ & = \text{knoten}\ l\ r.\end{aligned}$$

Der Anfang und die ersten beiden Schritte sind Rechnen nach den Definitionsgleichungen, also definitorisch. Erst der letzte Schritt braucht die beiden Induktionsvoraussetzungen $P\ l$ und $P\ r$.

Teil B: In Lean

Aufgabe B1 (Formalisierung von A2)

In der Datei steht das Urteil aus A2 (a) als Satz `kette`; dieselbe Typform für Daten ist die Definition `nacheinander`. Geben Sie beide Male den Term an. Zu A2 (c) ist `null` schon definiert, und zwar für jedes Universum `Sort u`. Ergänzen Sie `eins` und `zwei`, und zeigen Sie dann: `eins` $\mathbb{N}$ und `zwei` $\mathbb{N}$ zählen bis 1 und 2; `null` $\mathbb{N}$ und `zwei` $\mathbb{N}$ sind verschiedene Programme; für eine Aussage `A` sind `null A` und `zwei A` gleich. Die Ungleichung erfasst nur den Fall $\alpha := \mathbb{N}$: Hat α nur ein Element, so sind `null` und `zwei` dort dieselbe Funktion.

Hinweis: $a \neq b$ bedeutet $\neg(a = b)$, ein Beweis ist also eine Funktion, die aus $h : \text{null}\ \mathbb{N} = \text{zwei}\ \mathbb{N}$ einen Widerspruch macht. `congrFun h a` macht aus $h : f = g$ die Gleichung $f\,a = g\,a$; zweimal angewandt entsteht eine Gleichung zwischen zwei Zahlen. `absurd` schließt aus $e : P$ und $n : \neg P$ auf alles, und eine falsche Gleichung zwischen Zahlen widerlegt `by decide`.

Lean lean/Blatt02.lean

```
/-- Das Urteil aus A2 (a). Lean prueft genau
‘A : Prop, B : Prop, C : Prop ⊢ t : (A → B) → (B → C) → A → C‘. -/
theorem kette (A B C : Prop) : (A → B) → (B → C) → A → C :=
  sorry

/-- Derselbe Typ fuer Daten: erst ‘f‘ anwenden, dann ‘g‘. -/
def nacheinander (α β γ : Type) : (α → β) → (β → γ) → α → γ :=
  sorry

/- Die Terme aus A2 (c), fuer jedes Universum ‘Sort u‘. -/
universe u

def null (α : Sort u) : (α → α) → α → α := fun _ x => x

def eins (α : Sort u) : (α → α) → α → α :=
  sorry

def zwei (α : Sort u) : (α → α) → α → α :=
  sorry

/- Als Programme, mit α := ℕ: ‘eins‘ und ‘zwei‘ zaehlen bis 1 bzw. 2 ... -/
example : eins ℕ Nat.succ 0 = 1 :=
  sorry

example : zwei ℕ Nat.succ 0 = 2 :=
  sorry

/- ... und ‘null ℕ‘ ist ein anderes Programm als ‘zwei ℕ‘.
   ‘congrFun h a‘ macht aus ‘h : f = g‘ die Gleichung ‘f a = g a‘. -/
example : null ℕ ≠ zwei ℕ :=
  sorry

/- Als Beweise, mit einer Aussage A fuer α: -/
example (A : Prop) : null A = zwei A :=
  sorry
```

Lösung. `kette` und `nacheinander` bekommen denselben Term – einmal als Beweis gelesen, einmal als Programm, wie die beiden Lesarten von $\lambda a.\, \lambda b.\, a$ in der Vorlesung. Lean prüft dabei genau das Urteil aus A2 (a); den Baum baut der Kern selbst. `eins` und `zwei` sind die Terme aus A2 (c); angewandt auf `Nat.succ` und 0 rechnet der Kern sie zu 1 und 2 aus: `rfl`. Diese beiden Gleichungen prüfen zugleich Ihre Definitionen – mit dem Term von `null` für `eins` etwa fiele die erste durch.

Für die Ungleichung nimmt man `h` an und wendet beide Seiten mit `congrFun` auf `Nat.succ` und dann auf 0 an. Es entsteht eine Gleichung, deren Seiten zu 0 und 2 rechnen; `by decide` widerlegt sie, und `absurd` liefert daraus den verlangten Widerspruch. Die letzte Gleichung gilt per `rfl`: Für eine Aussage `A` sind beide Seiten Beweise derselben Aussage $(A \to A) \to A \to A$, und die Beweisirrelevanz macht sie definitorisch gleich.

Lean, Lösung loesungen/Blatt02_Loesung.lean

```
/-- Das Urteil aus A2 (a). Lean prueft genau
‘A : Prop, B : Prop, C : Prop ⊢ t : (A → B) → (B → C) → A → C‘. -/
theorem kette (A B C : Prop) : (A → B) → (B → C) → A → C :=
  fun f g x => g (f x)

/-- Derselbe Typ fuer Daten: erst ‘f‘ anwenden, dann ‘g‘. -/
```

```
def nacheinander (α β γ : Type) : (α → β) → (β → γ) → α → γ :=
  fun f g x => g (f x)

/- Die Terme aus A2 (c), fuer jedes Universum ‘Sort u‘. -/
universe u

def null (α : Sort u) : (α → α) → α → α := fun _ x => x

def eins (α : Sort u) : (α → α) → α → α :=
  fun f x => f x

def zwei (α : Sort u) : (α → α) → α → α :=
  fun f x => f (f x)

/- Als Programme, mit α := ℕ: ‘eins‘ und ‘zwei‘ zaehlen bis 1 bzw. 2 ... -/
example : eins ℕ Nat.succ 0 = 1 :=
  rfl

example : zwei ℕ Nat.succ 0 = 2 :=
  rfl

/- ... und ‘null ℕ‘ ist ein anderes Programm als ‘zwei ℕ‘.
   ‘congrFun h a‘ macht aus ‘h : f = g‘ die Gleichung ‘f a = g a‘. -/
example : null ℕ ≠ zwei ℕ :=
  fun h => absurd (congrFun (congrFun h Nat.succ) 0) (by decide)

/- Als Beweise, mit einer Aussage A fuer α: -/
example (A : Prop) : null A = zwei A :=
  rfl
```

Aufgabe B2 (Formalisierung von A3)

Die Aussagen aus A3 stehen in der Datei; gesucht sind Ihre Terme, in Lean geschrieben. Die Bausteine heißen dort `⟨_, _⟩` für das Paar, `h.1` und `h.2` für die Projektionen, `Or.inl` und `Or.inr` für die Injektionen und `h.elim` für die Fallunterscheidung; auch ein Beweis von ↔ ist ein Paar `⟨_, _⟩`. Für (d) steht `Classical.em A` zur Verfügung, ein Beweis von `A ∨ ¬A`. Lassen Sie sich danach mit `#print axioms` anzeigen, worauf `dreifach` und `klassisch` beruhen, und vergleichen Sie mit Abschnitt 3.4.

Lean `lean/Blatt02.lean`

```
/-- (a) Eine Implikation, angewandt auf die linke Komponente -/
theorem und_imp_links (A B C : Prop) : (A → B) → A ∧ C → B ∧ C :=
  sorry

/-- (b) De Morgan, in beiden Richtungen konstruktiv -/
theorem de_morgan (A B : Prop) : ¬(A ∨ B) ↔ ¬A ∧ ¬B :=
  sorry

/-- (c) Dreifache Negation -/
theorem dreifach (A : Prop) : ¬¬¬A → ¬A :=
  sorry

/-- (d) Die andere De-Morgan-Regel, mit ‘Classical.em A : A ∨ ¬A‘ -/
theorem klassisch (A B : Prop) : ¬(A ∧ B) → ¬A ∨ ¬B :=
  sorry

/- Worauf beruhen die beiden letzten Saetze? Ueber die Typentheorie hinaus
```

```
   braucht Mathlib nur ‘propext‘, ‘Classical.choice‘ und ‘Quot.sound‘. -/
#print axioms dreifach
#print axioms klassisch
```

Lösung. Die Terme sind die aus A3, in Lean geschrieben: λ wird `fun`, inl und inr werden `Or.inl` und `Or.inr`, die Fallunterscheidung wird `h.elim`. In (d) übernimmt `Classical.em A` die Rolle von em A, und im zweiten Fall genügt `Or.inl` selbst, ohne λ darum.

`#print axioms dreifach` meldet, dass der Satz von keinem Axiom abhängt: Er ist konstruktiv. Bei `klassisch` erscheinen alle drei Axiome `propext`, `Classical.choice` und `Quot.sound`. Der Satz vom ausgeschlossenen Dritten kommt nach Diaconescu aus der Auswahl, und mit ihm die klassische De-Morgan-Regel.

Lean, Lösung loesungen/Blatt02_Loesung.lean

```
/-- (a) Eine Implikation, angewandt auf die linke Komponente -/
theorem und_imp_links (A B C : Prop) : (A → B) → A ∧ C → B ∧ C :=
  fun f h => ⟨f h.1, h.2⟩

/-- (b) De Morgan, in beiden Richtungen konstruktiv -/
theorem de_morgan (A B : Prop) : ¬(A ∨ B) ↔ ¬A ∧ ¬B :=
  ⟨fun h => ⟨fun a => h (Or.inl a), fun b => h (Or.inr b)⟩,
   fun k h => h.elim k.1 k.2⟩

/-- (c) Dreifache Negation -/
theorem dreifach (A : Prop) : ¬¬¬A → ¬A :=
  fun h a => h (fun na => na a)

/-- (d) Die andere De-Morgan-Regel, mit ‘Classical.em A : A ∨ ¬A‘ -/
theorem klassisch (A B : Prop) : ¬(A ∧ B) → ¬A ∨ ¬B :=
  fun h => (Classical.em A).elim (fun a => Or.inr (fun b => h ⟨a, b⟩)) Or.inl

/- Worauf beruhen die beiden letzten Saetze? Ueber die Typentheorie hinaus
   braucht Mathlib nur ‘propext‘, ‘Classical.choice‘ und ‘Quot.sound‘. -/
#print axioms dreifach
#print axioms klassisch
```

Aufgabe B3 (Formalisierung von A4)

`zweimal` und `doppelt` sind wie in A4 definiert. Bestätigen Sie die Rechnungen aus A4 (a) und (b), und beweisen Sie die Gleichungen aus A4 (c): mit `rfl`, wo sie definitorisch gelten – dann rechnet der Kern selbst –, und sonst mit einem der Sätze `Nat.add_zero`, `Nat.zero_add`, `Nat.one_add`, `Nat.mul_zero`, `Nat.zero_mul` und `Nat.one_mul`, angewandt auf `n`. Was ein Satz besagt, zeigt `#check`. Nicht jeder passt; einer passt, obwohl er auf den ersten Blick nichts mit seiner Gleichung zu tun hat. Warum akzeptiert Lean ihn?

Lean lean/Blatt02.lean

```
def zweimal : (ℕ → ℕ) → ℕ → ℕ := fun f x => f (f x)
def doppelt : ℕ → ℕ := fun n => n + n

/- Die Rechnungen aus A4 (a) und (b) -/
example : zweimal doppelt 3 = 12 :=
  sorry

example : (let d := doppelt; d (d 1)) = 4 :=
  sorry
```

```
example : Nat.rec (motive := fun _ => ℕ) 1 (fun _ ih => 2 * ih) 3 = 8 :=
  sorry

/- Die Gleichungen aus A4 (c): 'rfl', wo sie definitorisch gelten, sonst
   einer der Saetze 'Nat.add_zero', 'Nat.zero_add', 'Nat.one_add',
   'Nat.mul_zero', 'Nat.zero_mul', 'Nat.one_mul' (nicht jeder passt). -/
example (n : ℕ) : n + 1 = Nat.succ n :=
  sorry

example (n : ℕ) : 1 + n = Nat.succ n :=
  sorry

example (n : ℕ) : 0 * n = 0 :=
  sorry

example (n : ℕ) : n * 1 = n :=
  sorry

example (n : ℕ) : n * 1 = 0 + n :=
  sorry
```

Lösung. Mit `rfl` gehen die drei Rechnungen durch und von den Gleichungen genau (i) und (v), die definitorischen aus A4 (c). Bei (ii), (iii) und (iv) lehnt Lean `rfl` mit einem Typfehler ab; dort helfen `Nat.one_add n`, `Nat.zero_mul n` und `Nat.zero_add n`. Die übrigen drei Sätze passen zu keiner Gleichung. Am nächsten scheint für (iv) `Nat.one_mul n` mit $1 \cdot n = n$ zu liegen, aber $1 \cdot n$ steckt an der Variablen n fest wie $0 \cdot n$ in (iii).

Der Satz `Nat.zero_add n` beweist eigentlich $0 + n = n$. Lean akzeptiert ihn als Beweis von $n \cdot 1 = n$ wegen der Konversionsregel: $n \cdot 1$ rechnet zu $0 + n$ (A4 (c) (iv)), also sind die Typen $0 + n = n$ und $n \cdot 1 = n$ definitorisch gleich, und ein Term des einen hat auch den anderen. In Leans Standardbibliothek ist `Nat.mul_one` übrigens genau so bewiesen.

Lean, Lösung `loesungen/Blatt02_Loesung.lean`

```
def zweimal : (ℕ → ℕ) → ℕ → ℕ := fun f x => f (f x)
def doppelt : ℕ → ℕ := fun n => n + n

/- Die Rechnungen aus A4 (a) und (b) -/
example : zweimal doppelt 3 = 12 :=
  rfl

example : (let d := doppelt; d (d 1)) = 4 :=
  rfl

example : Nat.rec (motive := fun _ => ℕ) 1 (fun _ ih => 2 * ih) 3 = 8 :=
  rfl

/- Die Gleichungen aus A4 (c): 'rfl', wo sie definitorisch gelten, sonst
   einer der Saetze 'Nat.add_zero', 'Nat.zero_add', 'Nat.one_add',
   'Nat.mul_zero', 'Nat.zero_mul', 'Nat.one_mul' (nicht jeder passt). -/
example (n : ℕ) : n + 1 = Nat.succ n :=
  rfl

example (n : ℕ) : 1 + n = Nat.succ n :=
  Nat.one_add n

example (n : ℕ) : 0 * n = 0 :=
  Nat.zero_mul n
```

```
example (n : ℕ) : n * 1 = n :=
  Nat.zero_add n    -- n * 1 rechnet zu 0 + n (Konversionsregel)
  -- ebenso ginge Nat.mul_one n

example (n : ℕ) : n * 1 = 0 + n :=
  rfl
```

Aufgabe B4 (Formalisierung von A5)

Die Datei enthält den Typ `Baum` und die Funktion `blaetter` aus A5.

(a) Definieren Sie `innen` und `spiegel` wie in A5, mit einem Fall pro Konstruktor nach dem Muster von `blaetter`.

(b) Bestätigen Sie mit `rfl` die Werte innen t und spiegel t aus A5 (a) und die Aussagen von A5 (c) und (d) für den Baum `beispiel`; das ist t aus A5 (a). Die ersten beiden Gleichungen prüfen zugleich Ihre Definitionen. Mit `#check @Baum.rec` können Sie Ihre Antwort zu A5 (b) vergleichen.

(c) Beweisen Sie `spiegel_spiegel`, also A5 (d) für alle Bäume.

Lean `lean/Blatt02.lean`

```
inductive Baum where
  | blatt : Baum
  | knoten : Baum → Baum → Baum

namespace Baum

/-- Anzahl der Blaetter: ein Fall pro Konstruktor, wie bei 'MeinNat.add'. -/
def blaetter : Baum → ℕ :=
  fun
  | .blatt => 1
  | .knoten l r => blaetter l + blaetter r

/-- (a) Anzahl der inneren Knoten -/
def innen : Baum → ℕ :=
  sorry

/-- (a) Spiegeln: linken und rechten Teilbaum tauschen, auf allen Ebenen -/
def spiegel : Baum → Baum :=
  sorry

/-- Der Baum 't' aus A5 (a) -/
def beispiel : Baum :=
  .knoten (.knoten .blatt .blatt) (.knoten .blatt (.knoten .blatt .blatt))

/- (b) Zuerst die Werte aus A5 (a) -- das prueft auch Ihre Definitionen. -/
example : innen beispiel = 4 :=
  sorry

example : spiegel beispiel =
    .knoten (.knoten (.knoten .blatt .blatt) .blatt) (.knoten .blatt .blatt) :=
  sorry

/- Dann A5 (c) und (d) am Beispiel -- der Kern rechnet beide Seiten aus. -/
example : blaetter beispiel = innen beispiel + 1 :=
```

```
  sorry

example : spiegel (spiegel beispiel) = beispiel :=
  sorry

#check @Baum.rec     -- zum Vergleich mit A5 (b)
```

Zu (c): Ein Induktionsbeweis ist in Lean eine rekursive Funktion wie `blaetter`; die Aufrufe `spiegel_spiegel l` und `spiegel_spiegel r` sind die Induktionsvoraussetzungen. `congrArg`$_2$ `knoten` macht aus $a = b$ und $c = d$ die Gleichung knoten $a\ c =$ knoten $b\ d$. A5 (c) für alle Bäume lassen wir weg: Dort braucht der Schritt Rechengesetze für $+$, bequem erst mit Taktiken wie `omega`.

Lean lean/Blatt02.lean

```
/-- (c) A5 (d) fuer alle Baeume. Im Fall '.knoten l r' sind
'spiegel_spiegel l' und 'spiegel_spiegel r' die Induktionsvoraussetzungen. -/
theorem spiegel_spiegel : (t : Baum) → spiegel (spiegel t) = t
  | .blatt =>
    sorry
  | .knoten l r =>
    sorry
end Baum
```

Lösung. `innen` und `spiegel` sind die Gleichungen aus A5; Lean akzeptiert die Rekursion, weil sie nur Teilbäume aufruft. Alle vier `rfl` in (b) rechnet der Kern aus. Die ersten beiden prüfen die Definitionen am Beispiel aus A5 (a): innen $t = 4$ und spiegel t wie dort gezeichnet. Ohne sie fiele ein `spiegel` nicht auf, das gar nicht tauscht: Auch die Identität erfüllt spiegel (spiegel t) $= t$, sogar mit demselben Beweis wie in (c). In den anderen beiden ergibt sich links 5 und rechts $4 + 1$ sowie spiegel (spiegel t) $\rightsquigarrow t$.

Lean, Lösung loesungen/Blatt02_Loesung.lean

```
inductive Baum where
  | blatt : Baum
  | knoten : Baum → Baum → Baum

namespace Baum

/-- Anzahl der Blaetter: ein Fall pro Konstruktor, wie bei 'MeinNat.add'. -/
def blaetter : Baum → ℕ :=
  fun
  | .blatt => 1
  | .knoten l r => blaetter l + blaetter r

/-- (a) Anzahl der inneren Knoten -/
def innen : Baum → ℕ :=
  fun
  | .blatt => 0
  | .knoten l r => innen l + innen r + 1

/-- (a) Spiegeln: linken und rechten Teilbaum tauschen, auf allen Ebenen -/
def spiegel : Baum → Baum :=
  fun
  | .blatt => .blatt
  | .knoten l r => .knoten (spiegel r) (spiegel l)

/-- Der Baum 't' aus A5 (a) -/
def beispiel : Baum :=
```

```
  .knoten (.knoten .blatt .blatt) (.knoten .blatt (.knoten .blatt .blatt))

/- (b) Zuerst die Werte aus A5 (a) -- das prueft auch Ihre Definitionen. -/
example : innen beispiel = 4 :=
  rfl

example : spiegel beispiel =
    .knoten (.knoten (.knoten .blatt .blatt) .blatt) (.knoten .blatt .blatt) :=
  rfl

/- Dann A5 (c) und (d) am Beispiel -- der Kern rechnet beide Seiten aus. -/
example : blaetter beispiel = innen beispiel + 1 :=
  rfl

example : spiegel (spiegel beispiel) = beispiel :=
  rfl

#check @Baum.rec    -- zum Vergleich mit A5 (b)
```

Im Beweis von `spiegel_spiegel` ist der Fall `.blatt` reine Rechnung. Im Fall `.knoten l r` rechnet der Kern die linke Seite zu knoten (spiegel (spiegel l)) (spiegel (spiegel r)) aus – die definitorischen Schritte aus A5 (d), erledigt von der Konversionsregel –, und $\texttt{congrArg}_2$ `knoten` setzt die Induktionsvoraussetzungen ein. Derselbe Rekursor erzeugt Funktionen und Beweise.

Lean, Lösung loesungen/Blatt02_Loesung.lean

```
/-- (c) A5 (d) fuer alle Baeume. Im Fall `.knoten l r` sind
`spiegel_spiegel l` und `spiegel_spiegel r` die Induktionsvoraussetzungen. -/
theorem spiegel_spiegel : (t : Baum) → spiegel (spiegel t) = t
  | .blatt =>
    rfl
  | .knoten l r =>
    congrArg₂ knoten (spiegel_spiegel l) (spiegel_spiegel r)
end Baum
```